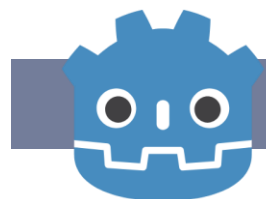


게임엔진 기초

Godot Shader 사용해 보기

강영민

동명대학교 게임그래픽학과



GODOT

Shader를 이용한 렌더링 이해

Shader

그래픽 처리 장치(GPU)에서 작동되는 특별한 프로그램

초기 Shader

3D 씬의 음영 계산을 위해 사용

현재의 Shader

다양한 지오메트리와 픽셀 조작

일반적인 계산 작업도 수행 → 범용 GPU 개념 등장

핵심 개념

병렬 처리를 이용한 고속 렌더링

```
for x in range(width):  
    for y in range(height):  
        set_color(x, y, some_color)
```

GDScript



```
void fragment() {  
    COLOR = some_color;  
}
```

Shader

The graphics card calls the **fragment()** function once or more for each pixel it has to draw. More on that below.

Godot Shader

OpenGL의 표준 Shader 언어인 GLSL에 기반 → 간략화

- 1.The **vertex()** function runs over all the vertices in the mesh and sets their positions and some other per-vertex variables. Used in [canvas item shaders](#) and [spatial shaders](#).
- 2.The **fragment()** function runs for every pixel covered by the mesh. It uses values output by the **vertex()** function, interpolated between the vertices. Used in [canvas item shaders](#) and [spatial shaders](#).
- 3.The **light()** function runs for every pixel and for every light. It takes variables from the **fragment()** function and from its previous runs. Used in [canvas item shaders](#) and [spatial shaders](#).
- 4.The **start()** function runs for every particle in a particle system once when the particle is first spawned. Used in [particles shaders](#).
- 5.The **process()** function runs for every particle in a particle system for each frame. Used in [particles shaders](#).
- 6.The **sky()** function runs for every pixel in the radiance cubemap when the radiance cubemap needs to be updated, and for every pixel on the current screen. Used in [sky shaders](#).
- 7.The **fog()** function runs for every froxel in the volumetric fog froxel buffer that intersects with the [FogVolume](#). Used by [fog shaders](#).

Shader 종류

- [_spatial](#) for 3D rendering
- [_canvas_item](#) for 2D rendering
- [_particles](#) for particle systems
- [_sky](#) to render [Skies](#)
- [_fog](#) to render [FogVolumes](#)

Render mode

- shaded / unshaded
- culling enabled or disabled

Shader의 첫 두 줄은 Shader 타입과 렌더 모드를 지정

```
shader_type spatial;  
render_mode unshaded, cull_disabled;
```



일반적으로 흔히 사용되는 처리기

Vertex processor

```
shader_type spatial;  
render_mode skip_vertex_transform;
```

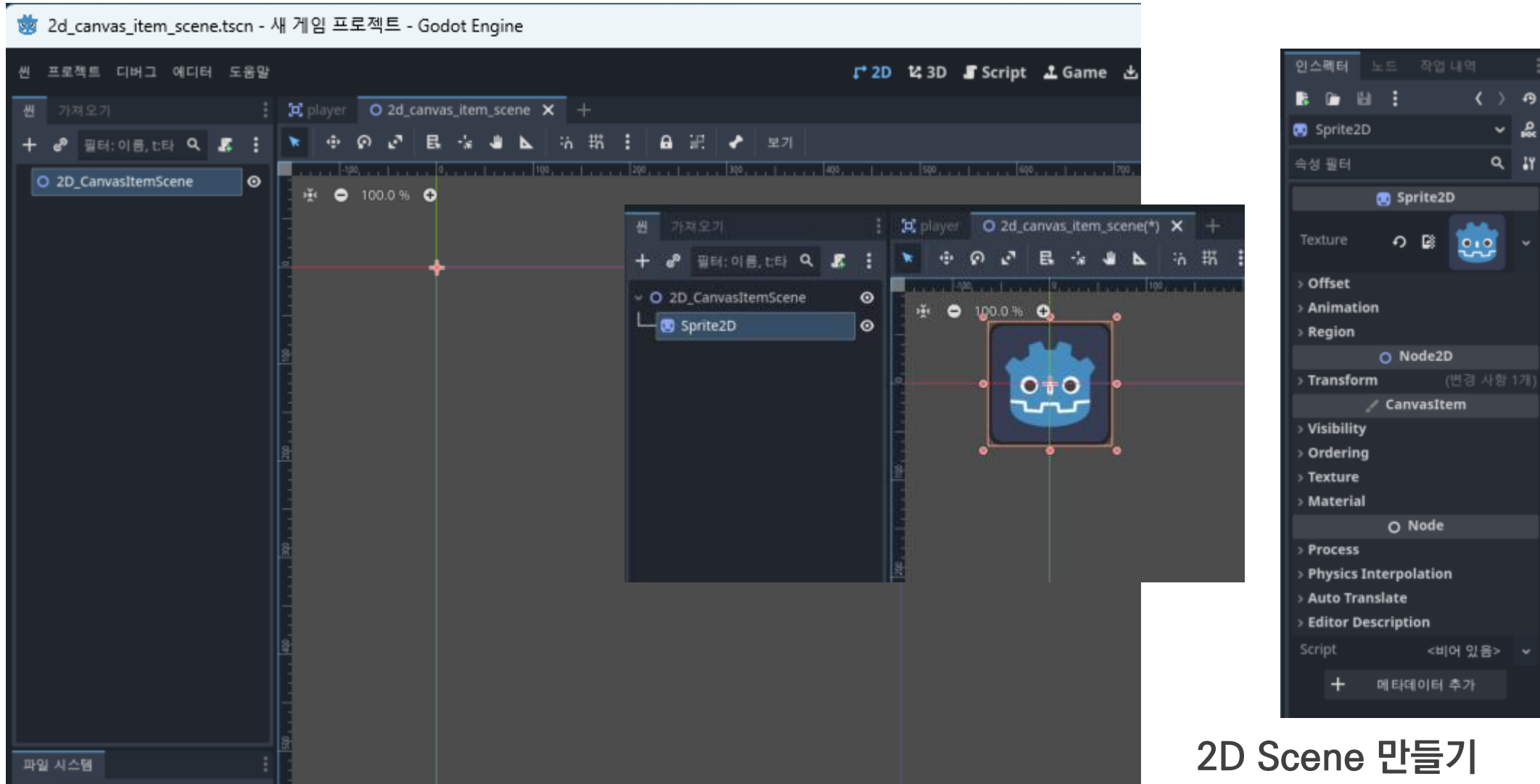
Fragment processor

```
void vertex() {  
    VERTEX = (MODELVIEW_MATRIX * vec4(VERTEX, 1.0)).xyz;  
    NORMAL = normalize((MODELVIEW_MATRIX * vec4(NORMAL, 0.0)).xyz);  
    BINORMAL = normalize((MODELVIEW_MATRIX * vec4(BINORMAL, 0.0)).xyz);  
    TANGENT = normalize((MODELVIEW_MATRIX * vec4(TANGENT, 0.0)).xyz);  
}
```

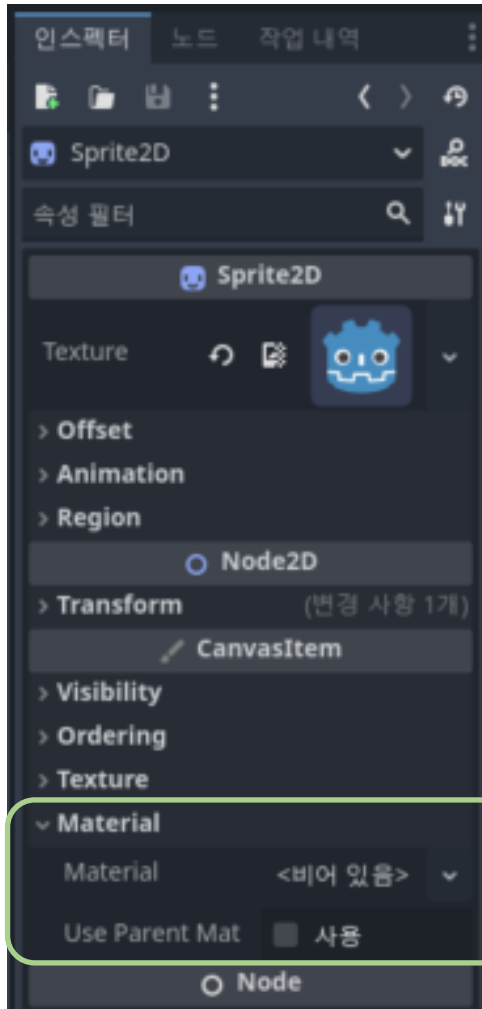
Light processor

복잡한 문법 이야기는 일단 미루어 두자...

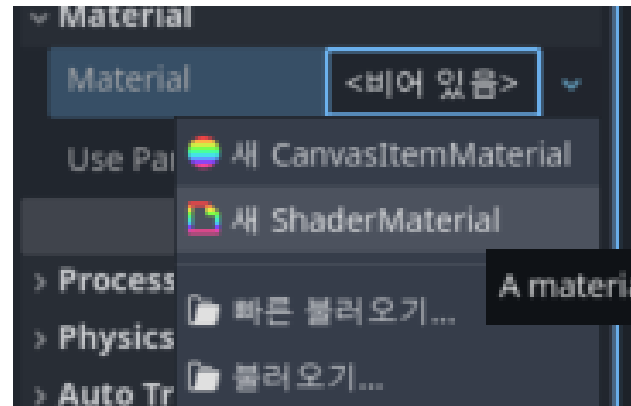
2D CanvasItem Shader를 만들어 보자



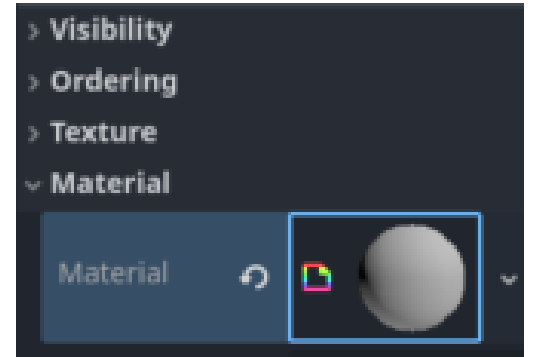
재질(Material) 살펴보기



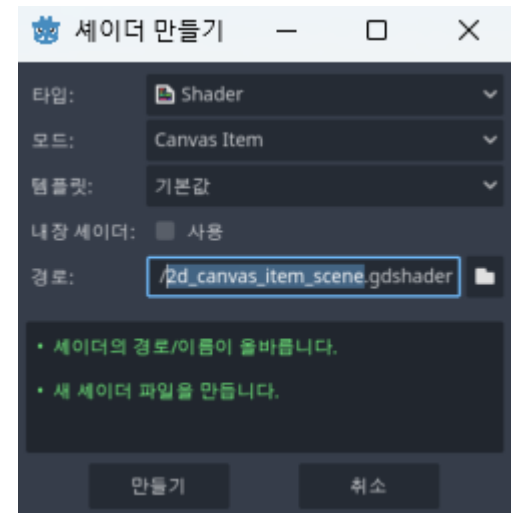
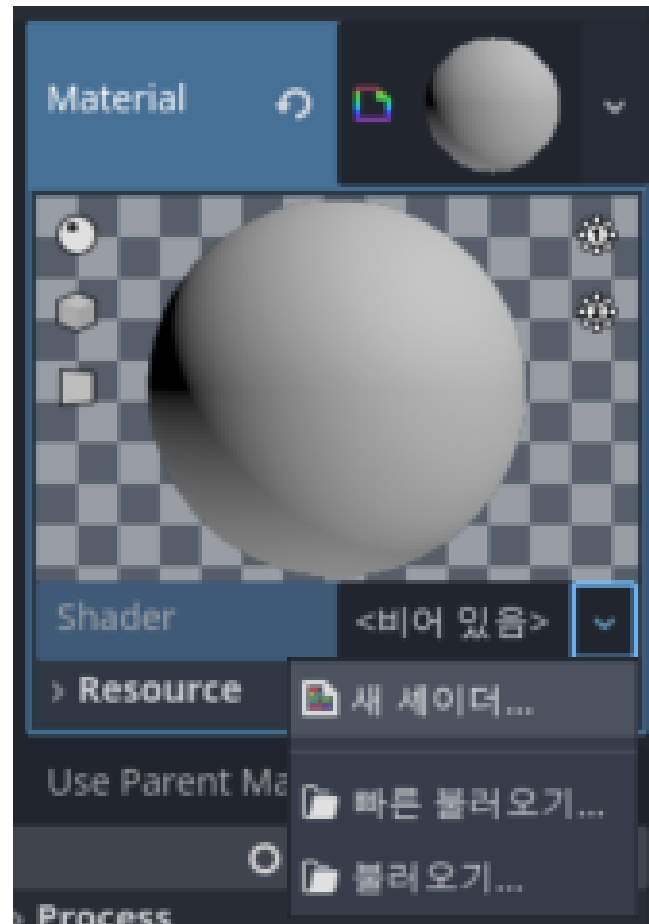
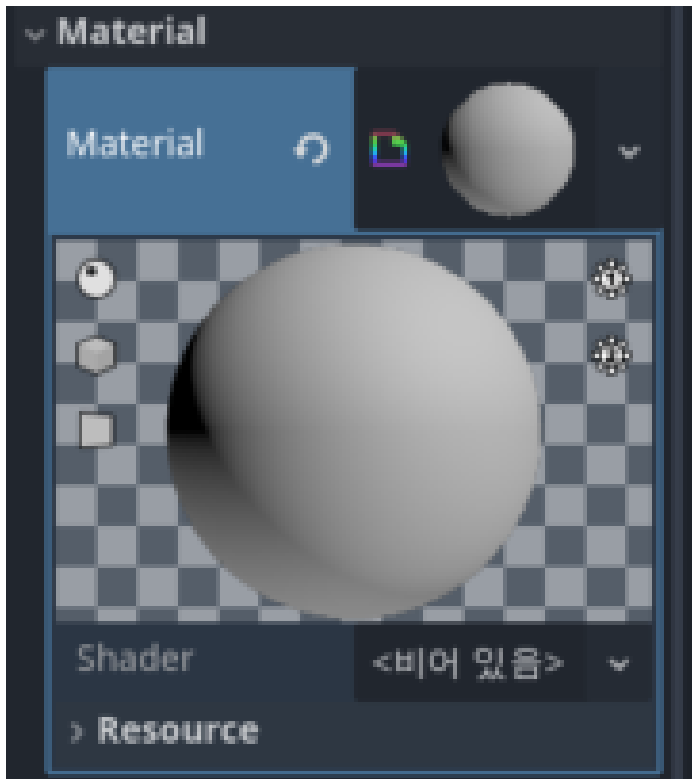
비어있음



새 셰이더 추가



Shader 코드 작성



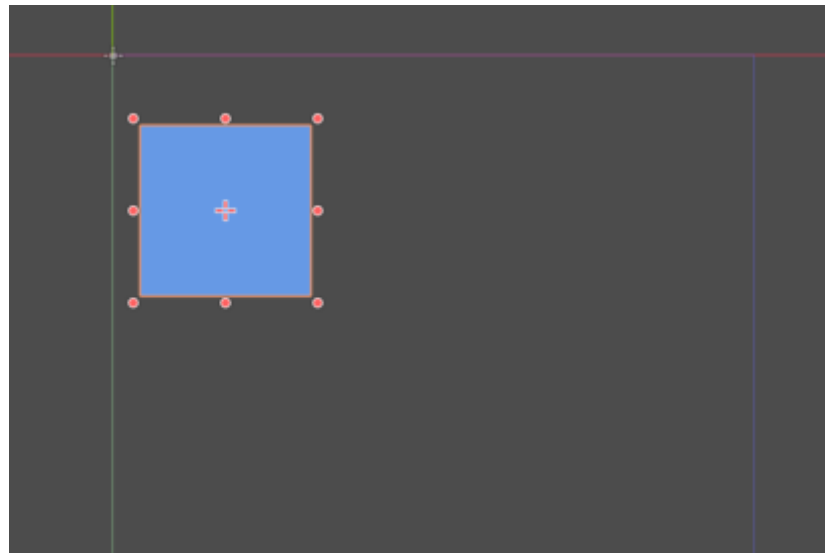
새 셰이더 추가

생성된 Shader

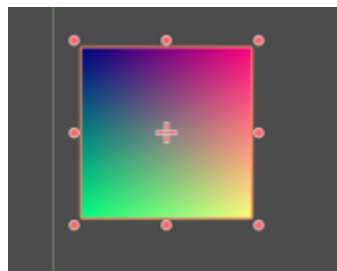
```
2d_canvas_item_scene....  
1  shader_type canvas_item;  
2  
3  void vertex() {  
4      // Called for every vertex the material is visible on.  
5  }  
6  
7  void fragment() {  
8      // Called for every pixel the material is visible on.  
9  }  
10  
11  //void light() {  
12      // Called for every pixel for every light affecting the CanvasItem.  
13      // Uncomment to replace the default light processing function with this one.  
14  }  
15
```

고쳐보자

```
void fragment(){  
    COLOR = vec4(0.4, 0.6, 0.9, 1.0);  
}
```

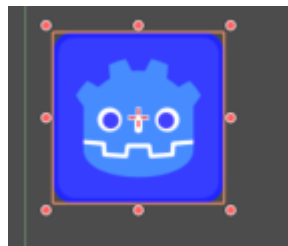


```
void fragment() {  
    COLOR = vec4(UV, 0.5, 1.0);  
}
```



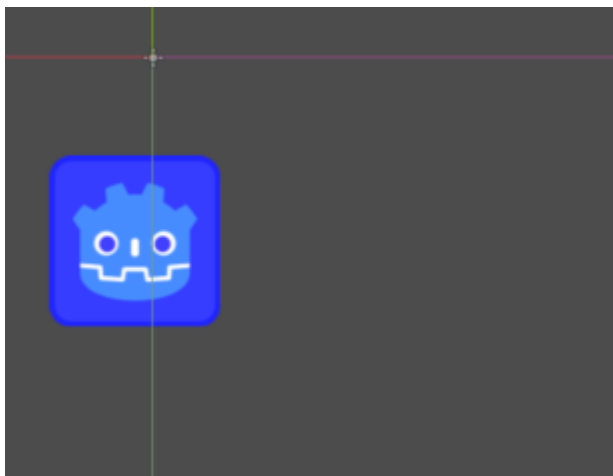
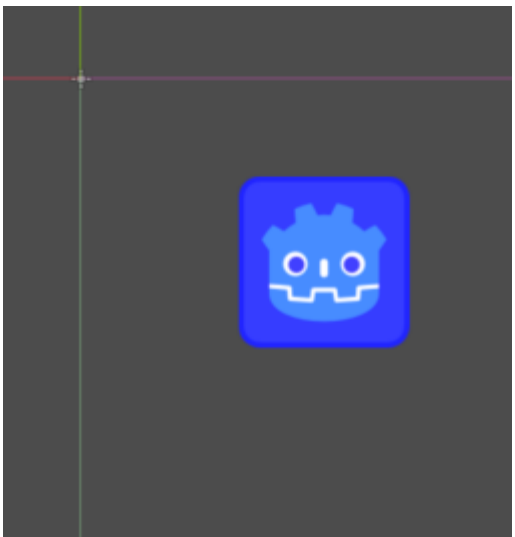
플레이어 씬 새로 만들기

```
void fragment(){  
    // This shader will result in a blue-tinted icon  
    COLOR.b = 1.0;  
}
```

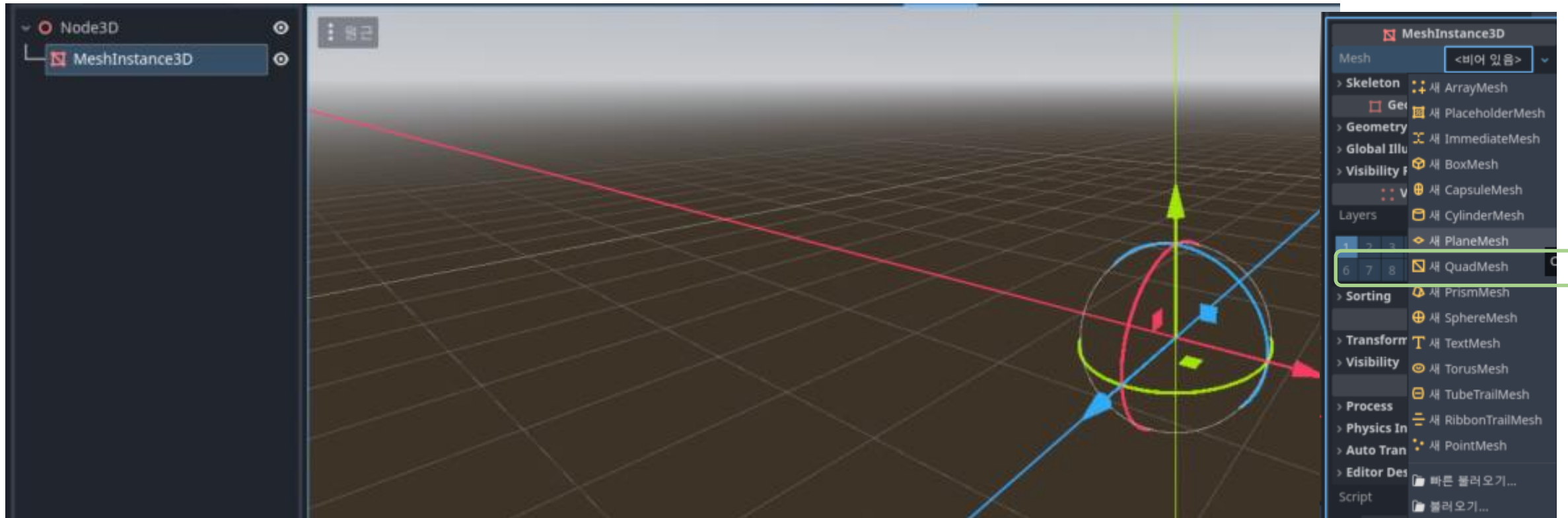


vertex 애니메이션

```
void vertex() {  
    // Animate Sprite2D moving in big circle around its location  
    VERTEX += vec2(cos(TIME)*100.0, sin(TIME)*100.0);  
}
```



3차원 Shader

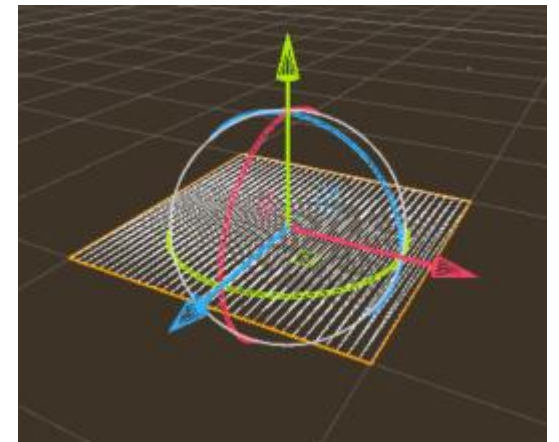
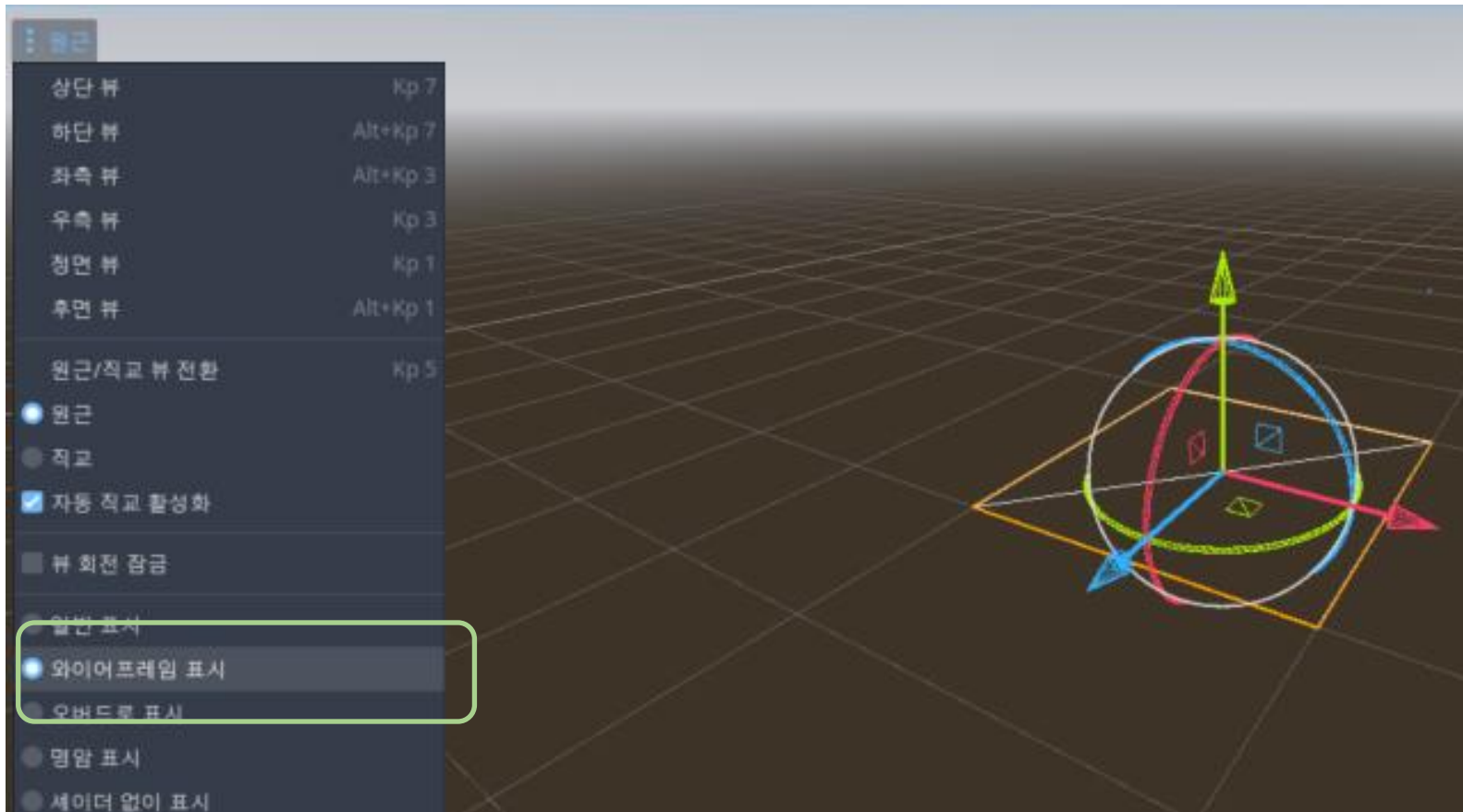


MeshInstance3D 추가

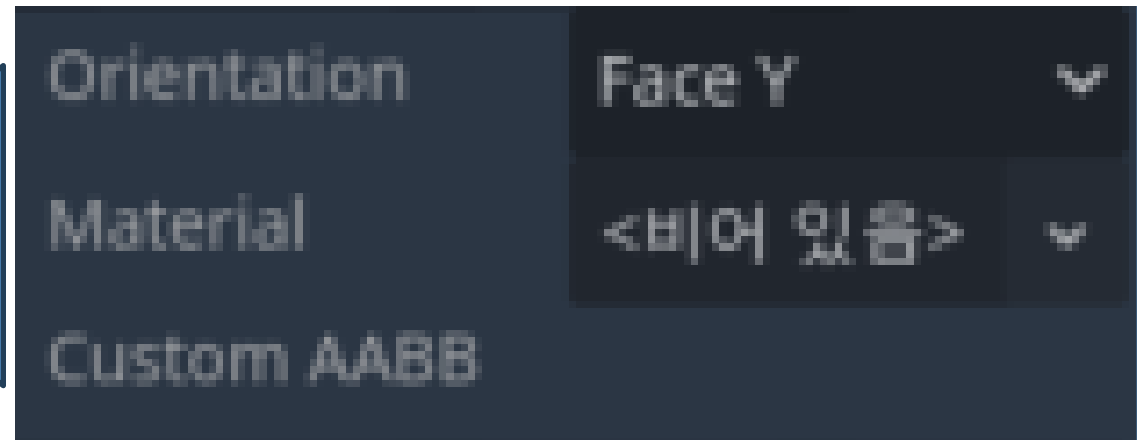
메시 보기

Wireframe 보기기

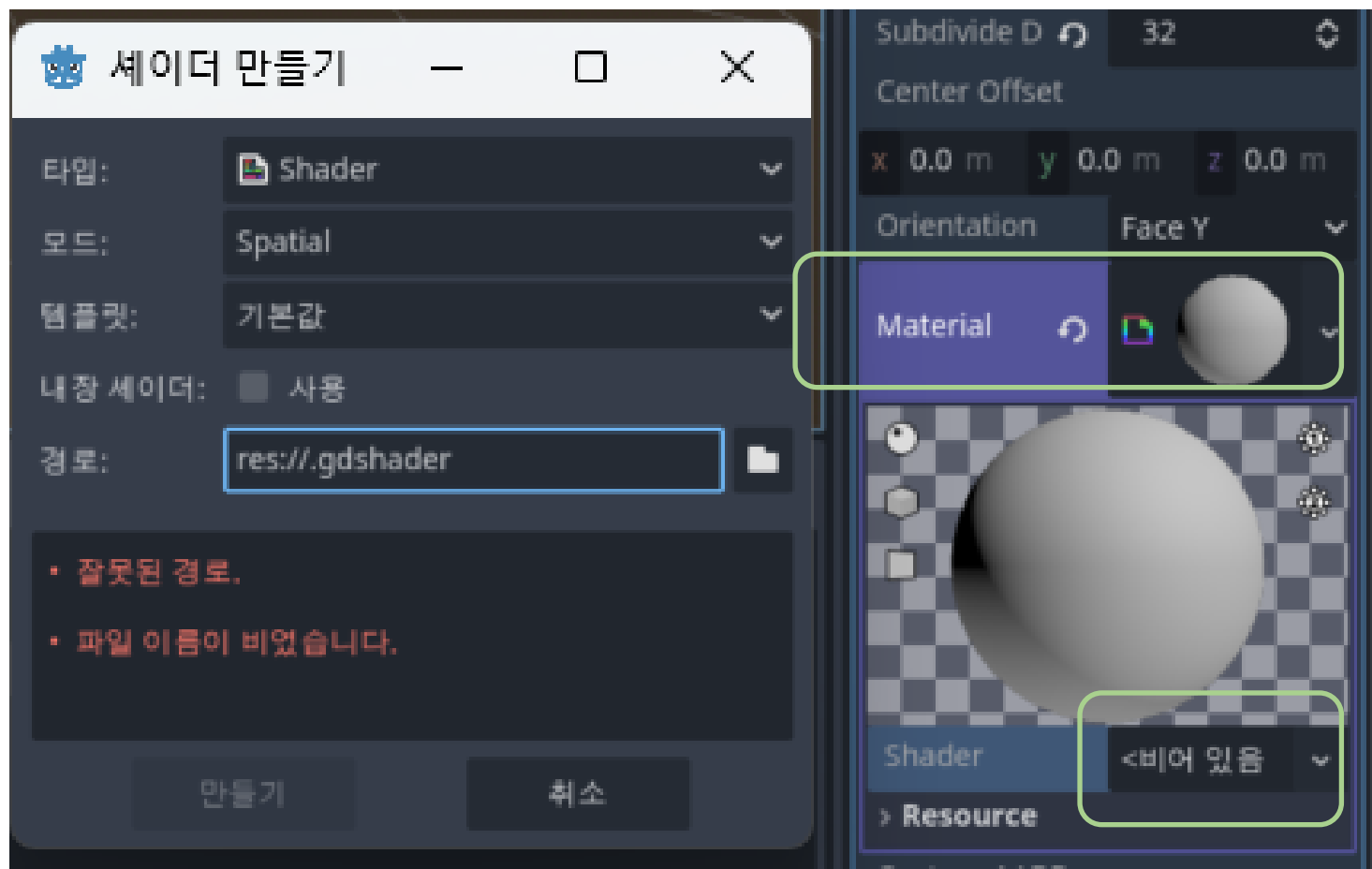
→ 메시 쪼개기 (subdivision)



Material

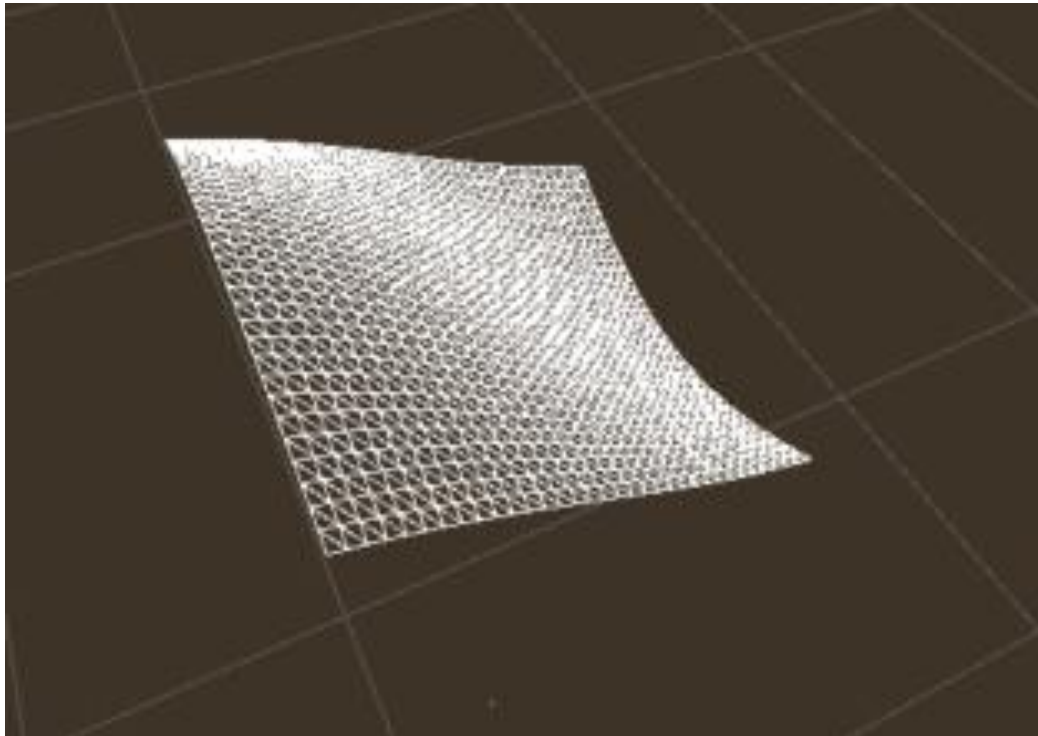


새로운 Shader 추가



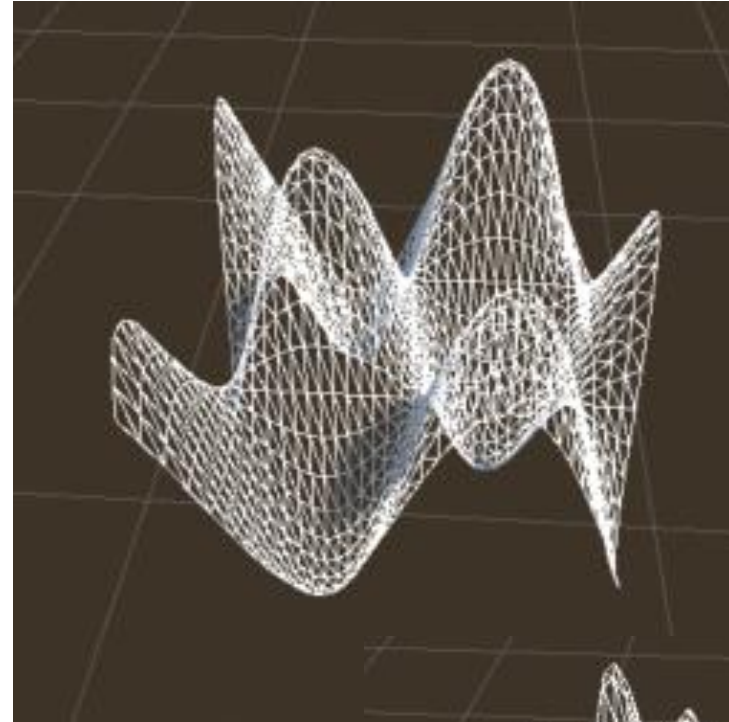
Shader Magic

```
void vertex() {  
    VERTEX.y += cos(VERTEX.x) * sin(VERTEX.z);  
}
```

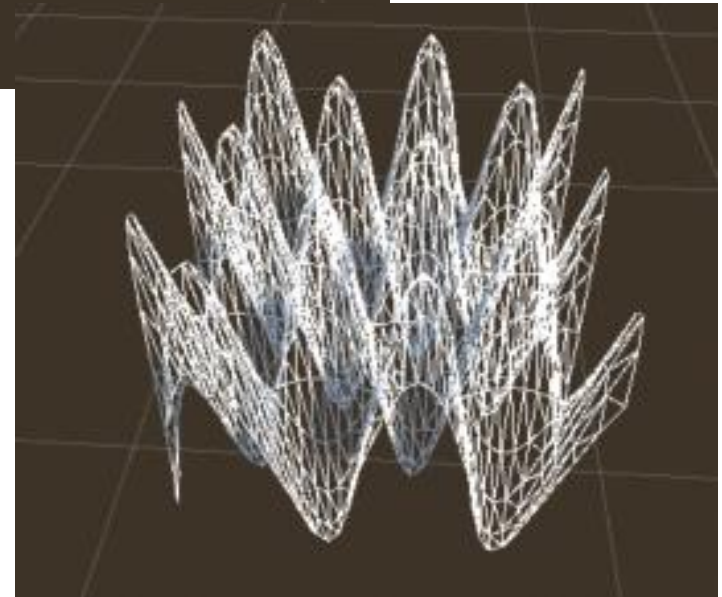


Shader 조작해 보기

```
void vertex() {  
    VERTEX.y += cos(VERTEX.x * 4.0) * sin(VERTEX.z * 4.0);  
}
```



```
void vertex() {  
    VERTEX.y += cos(VERTEX.x * 4.0) * sin(VERTEX.z * 4.0);  
}
```



노이즈 추가

```
shader_type spatial;
```

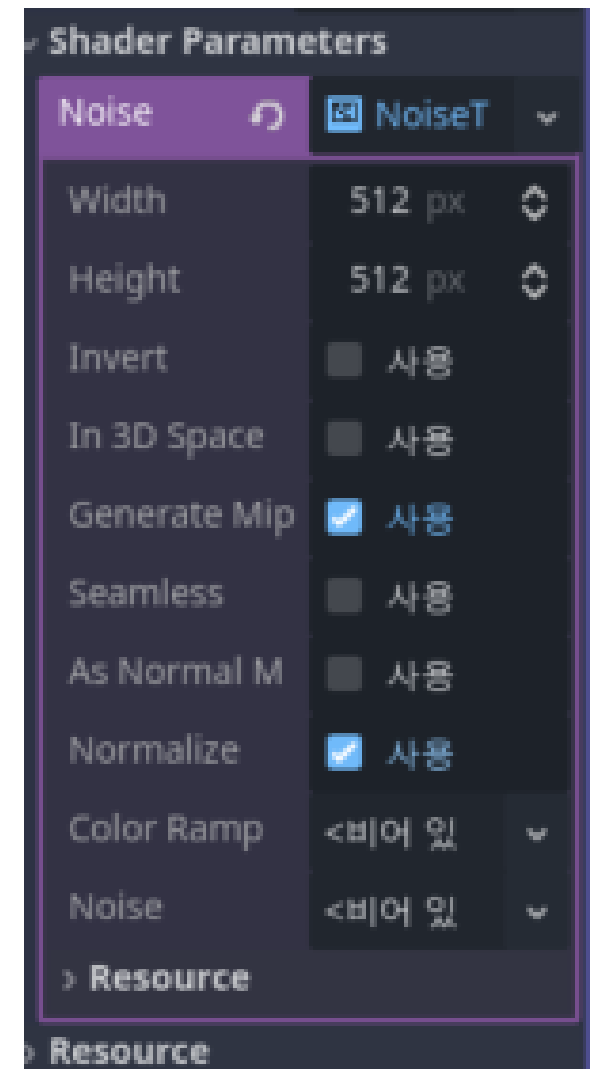
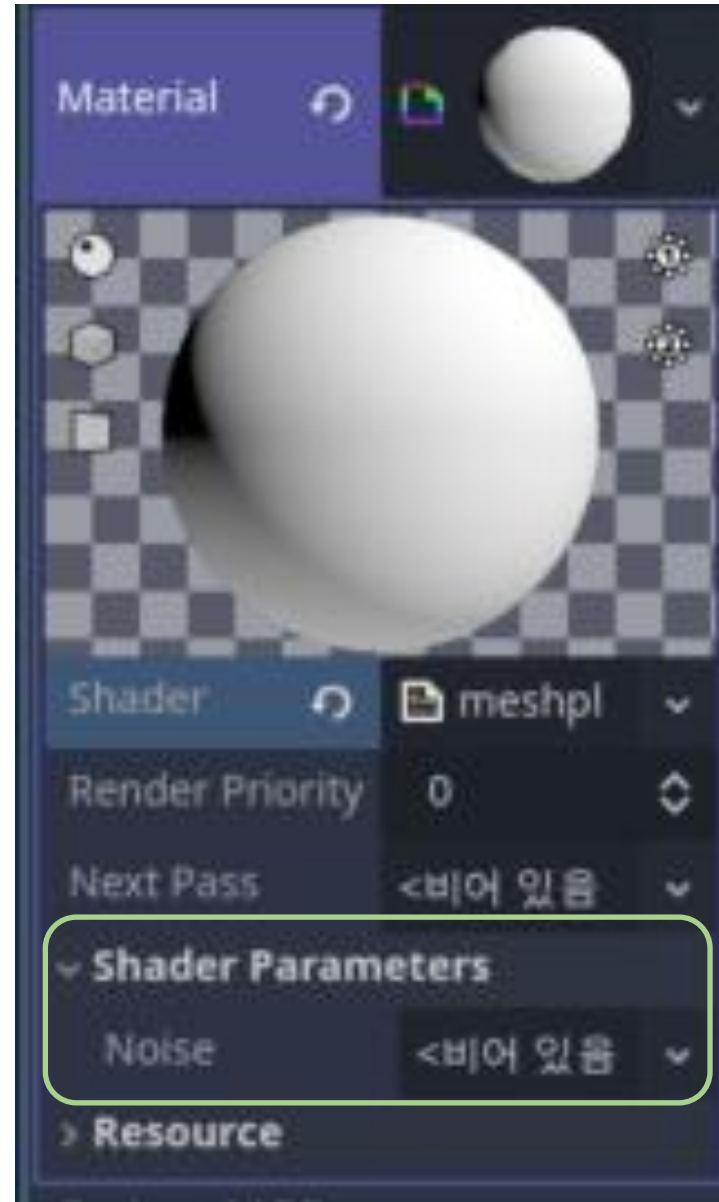
```
uniform sampler2D noise;
```

```
void vertex() {
```

```
}
```

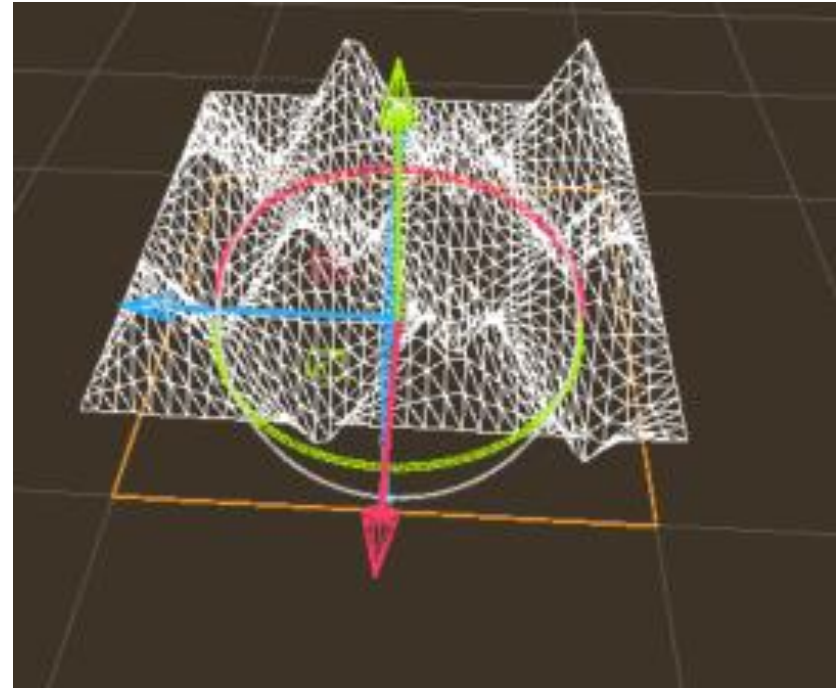
```
void fragment() {
```

```
}
```



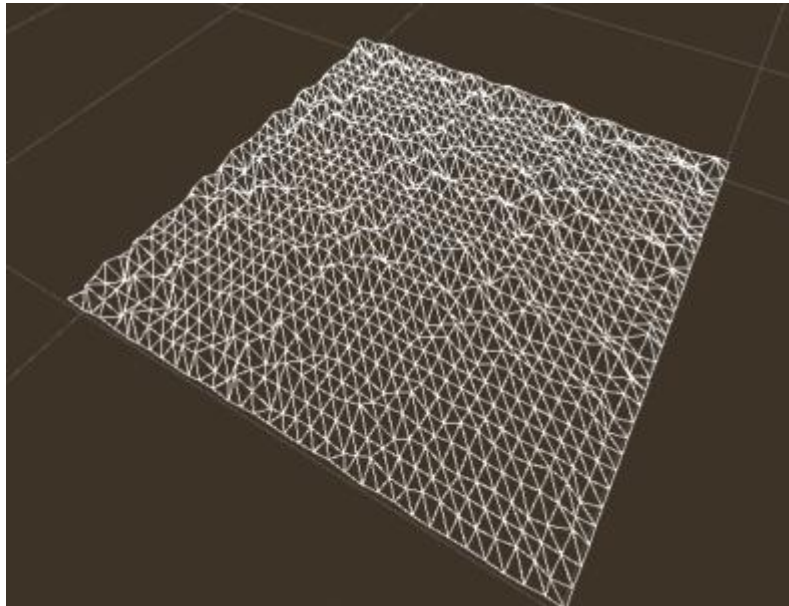
노이즈 가져오기 – texture

```
void vertex() {  
    float height = texture(noise, VERTEX.xz / 2.0 + 0.5).x;  
    VERTEX.y += height;  
}
```



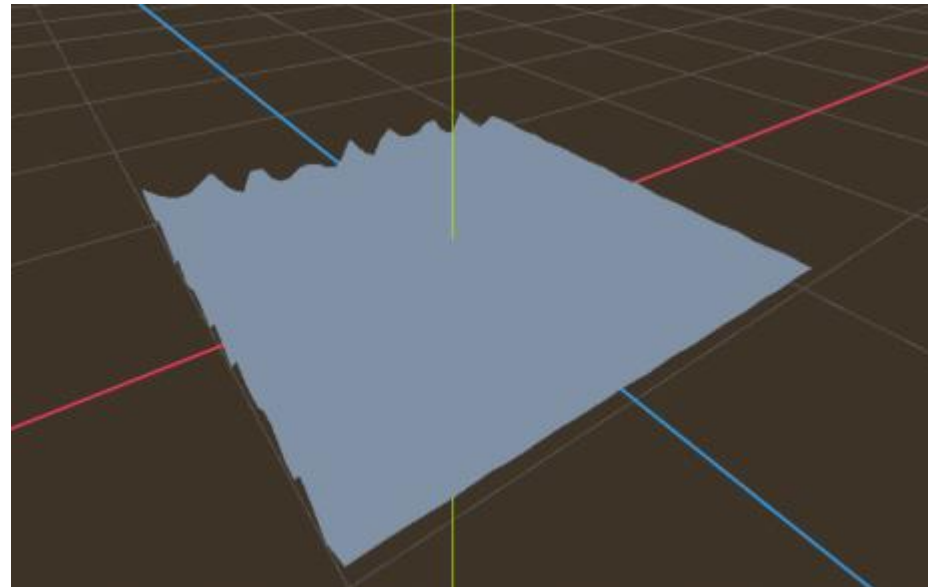
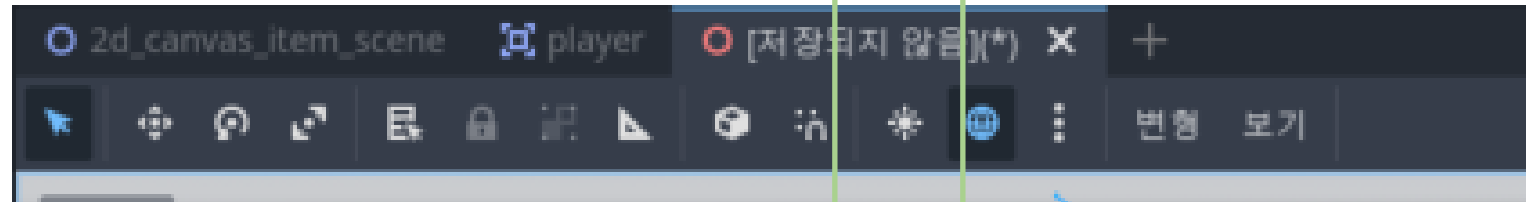
애니메이션

```
void vertex() {  
    float height = 0.2 * texture(noise, cos(5.0*VERTEX.xz + TIME*0.2) / 2.0 + 0.5).x;  
    VERTEX.y += height;  
}
```

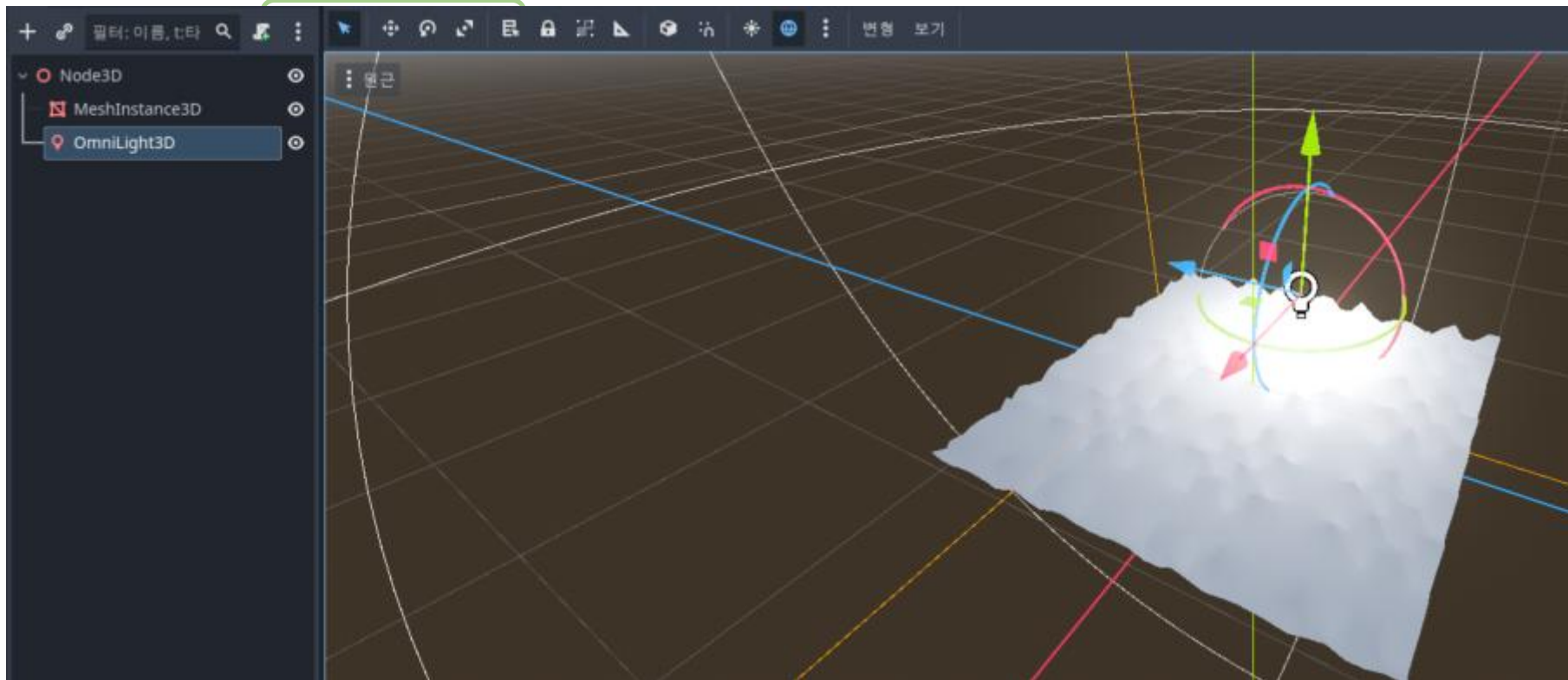


라이팅

Preview Sunlight 켜기



라이트 추가



법선 맵 추가

```
shader_type spatial;
```

```
uniform sampler2D noise;
```

```
uniform sampler2D normalmap;
```

```
varying vec2 tex_position;
```

```
void vertex() {
```

```
    tex_position = VERTEX.xz / 2.0 + 0.5;
```

```
    float height = 0.2 * texture(noise, cos(5.0*VERTEX.xz + TIME*0.2) / 2.0 + 0.5).x;
```

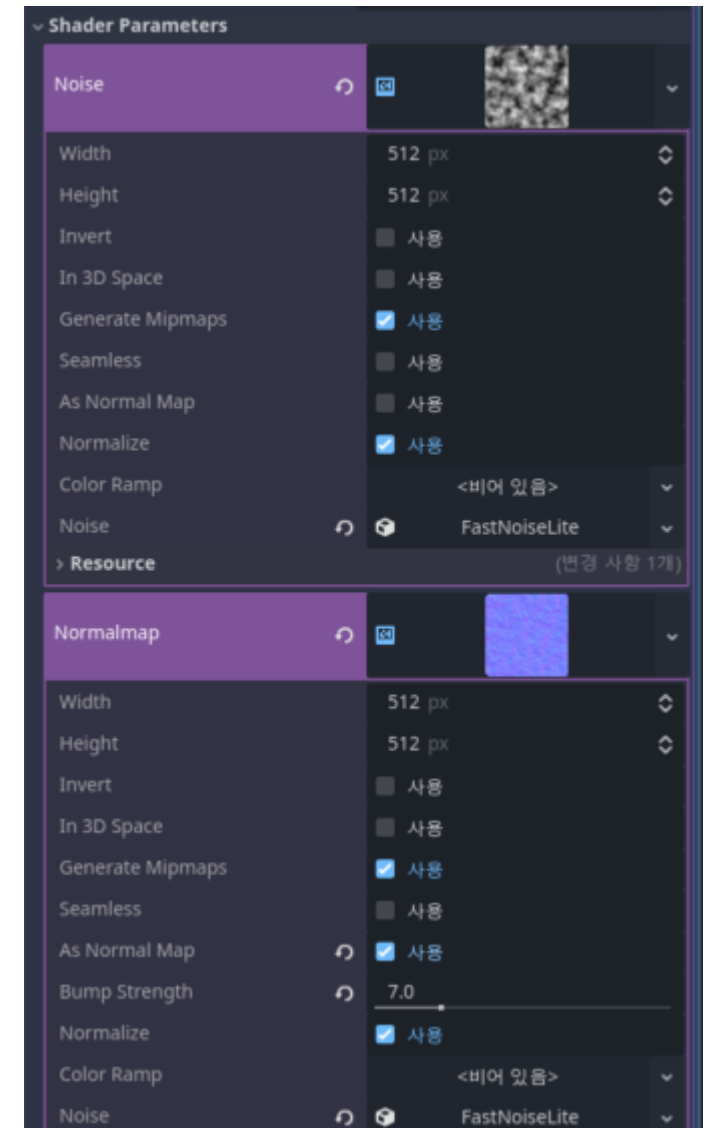
```
    VERTEX.y += height;
```

```
}
```

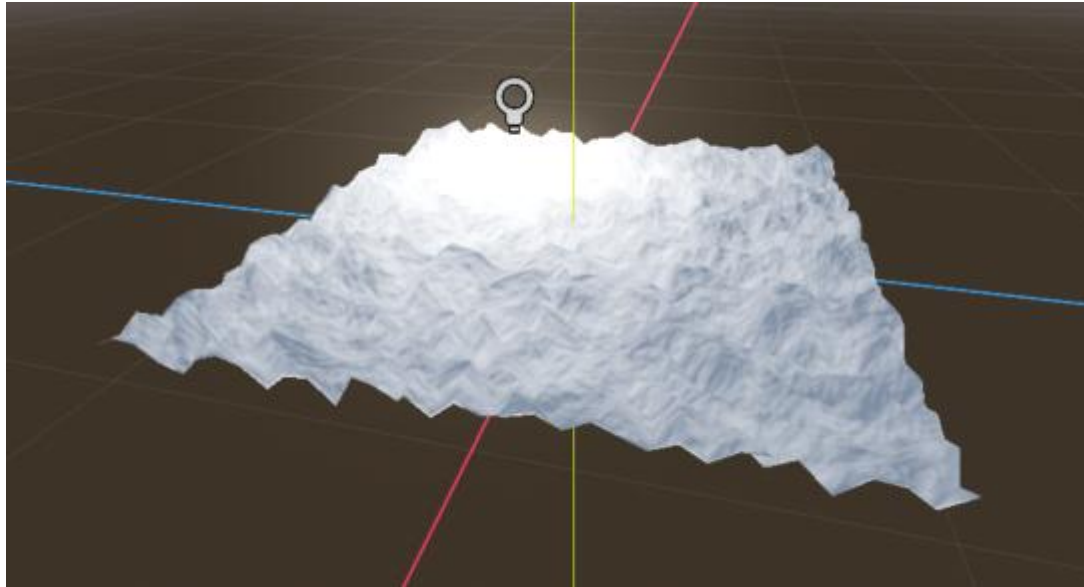
```
void fragment() {
```

```
    NORMAL_MAP = texture(normalmap, tex_position).xyz;
```

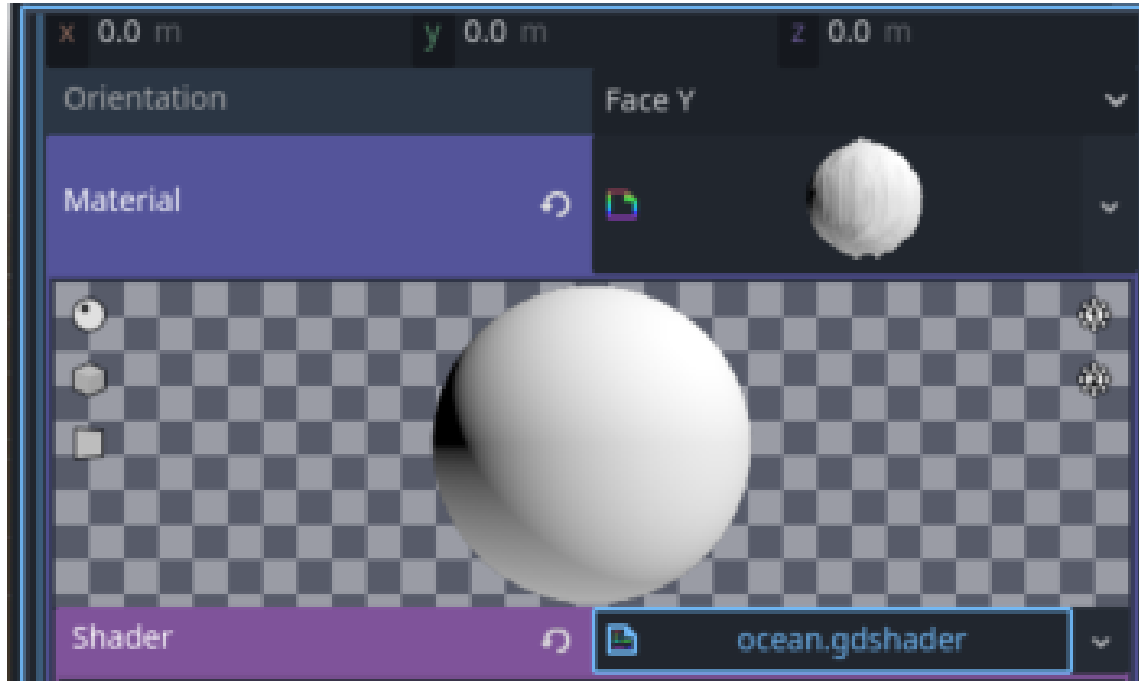
```
}
```



라이트에 반응하는 디테일



새로운 Shader



파도 생성

```
shader_type spatial;  
render_mode specular_toon;
```

```
uniform sampler2D noise;  
uniform sampler2D normalmap;
```

```
float wave(vec2 position) {  
    position += texture(noise, position / 10.0).x * 2.0 - 1.0;  
    vec2 wv = 1.0 - abs(sin(position));  
    return pow(1.0 - pow(wv.x * wv.y, 0.65), 4.0);  
}
```

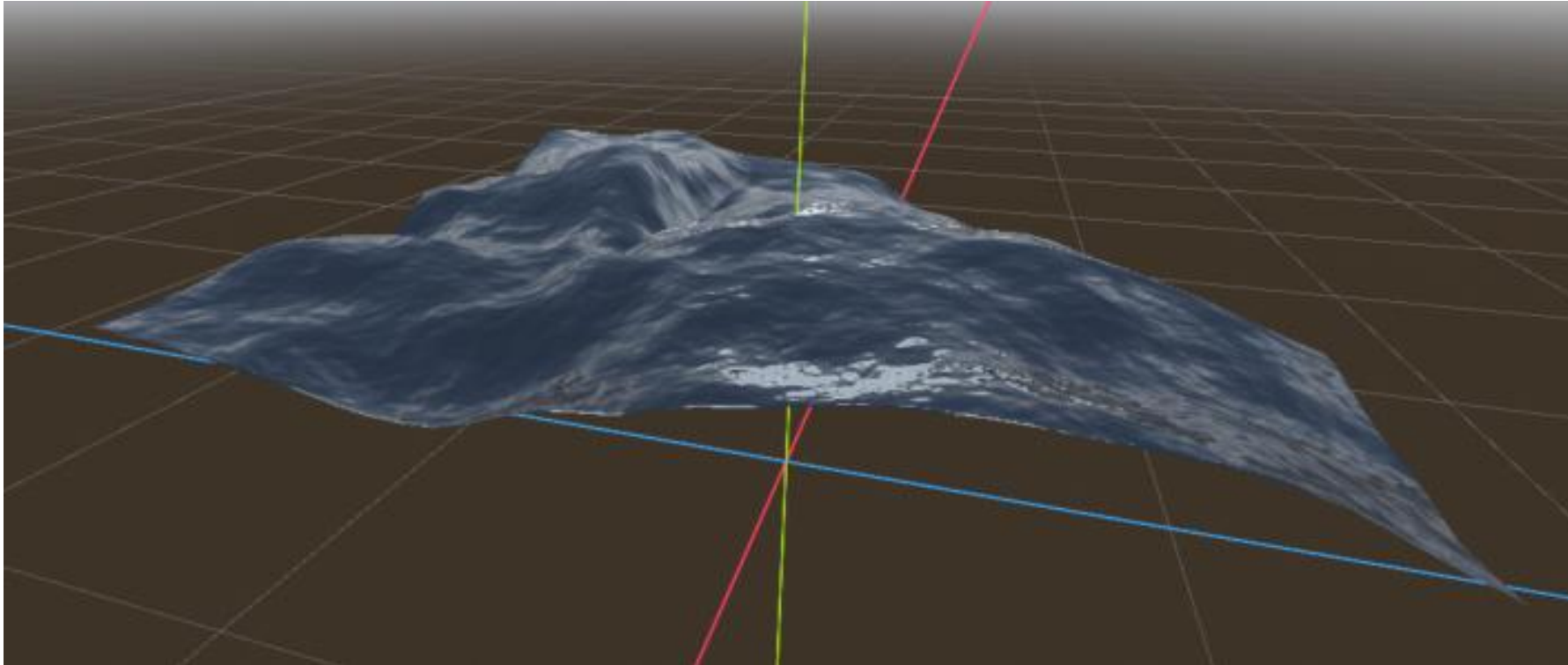
```
float height(vec2 position, float time) {  
    // Change your MeshInstance3D PlaneMesh to 10 x 10  
  
    float d = wave((position + time) * 0.4) * 0.3;  
  
    d += wave((position - time) * 0.3) * 0.3;  
    d += wave((position + time) * 0.5) * 0.2;  
    d += wave((position - time) * 0.6) * 0.2;  
    return d;  
}
```

정점과 프래그먼트 설정

```
void vertex() {  
    vec2 pos = VERTEX.xz;  
    float k = height(pos, TIME);  
  
    // Version - 2  
    VERTEX.y = k;  
    NORMAL = normalize(vec3(k - height(pos + vec2(0.1, 0.0), TIME), 0.1, k - height(pos + vec2(0.0, 0.1), TIME)));  
}  
  
void fragment() {  
  
    float fresnel = sqrt(1.0 - dot(NORMAL, VIEW));  
    NORMAL_MAP = texture(normalmap, UV).xyz;  
  
    RIM = 0.2;  
    METALLIC = 0.0;  
    ROUGHNESS = 0.01 * (1.0 - fresnel);  
    ALBEDO = vec3(0.01, 0.03, 0.05) + (0.1 * fresnel);  
}
```

정점과 프래그먼트 설정

Voila!





Questions?